

系统开发工具基础

第三周实验报告

乔海轩—25090012035

2026 年 9 月 7 日

目录

1 总览	2
2 第 9 题从源码构建并在干净环境安装 Wheel	3
3 第 10 题让编程智能体进入可验证的修复循环	4
4 第 11 题把“无法处理”的协作材料改成可执行信息	6
5 第 12 题修复一个可复现的线性回归训练循环	8

1 总览

题目编号	描述	完成情况
9	从源码构建并在干净环境安装 Wheel	已完成
10	让编程智能体进入可验证的修复循环	已完成
11	把“无法处理”的协作材料改成可执行信息	已完成
12	修复一个可复现的线性回归训练循环	已完成

Github 仓库地址: <https://github.com/Ptilotsis-o/system-dev-tools/>
Commit 记录如图 1 所示。

```
* 87956a4 feat(q12): finished
* 8a46b99 feat(q12): file prepared
* 1e4cd24 feat(q11): rewrite the doc
* 78fb1bf feat(q10): write ai_log.md
* d2f85d9 feat(q10): fixed cli.py
* cb83519 feat(q10): craete test file
* babffea chore: ignore python's build/
* 6dc3913 feat(q09): files prepared
```

图 1: Commit 记录截图

2 第 9 题从源码构建并在干净环境安装 Wheel

题目要求

- 创建 `src/greetlab/__init__.py`、给定的 `cli.py` 和 `pyproject.toml`，并把“学号”替换为自己的学号。
- 在已安装 `build` 工具的课程环境中运行 `python -m build`，生成 `wheel`。
- 新建一个干净虚拟环境，只从生成的 `wheel` 安装。
- 切换到 `q09` 之外运行 `sdt-greet --name < 学号 >`，确认输出正确。

操作步骤与命令

```
mkdir -p q09/src/greetlab
vim q09/src/greetlab/__init__.py
vim q09/src/greetlab/cli.py
vim q09/pyproject.toml
cd q09
python -m build
python -m venv env
source env/bin/activate
pip install dist/greetlab_25090012035-0.1.0-py3-none-any.whl
cd ..
sdt-greet --name 25090012035
```

结果验证

- 三个文件创建完成，`pyproject.toml` 中的包名为 `greetlab-25090012035`；
- `python -m build` 执行成功，`dist/` 下生成 `greetlab_25090012035-0.1.0-py3-none-any.whl`；
- 新建的虚拟环境只安装了该 `wheel`，未从源码路径安装；
- 在 `q09` 之外运行 `sdt-greet --name 25090012035`，输出 `Hello, 25090012035!`。

```
(env) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week3$ sdt-greet --name 25898012035
Hello, 25898012035!
```

图 2: 在 q09 之外运行 sdt-greet 输出截图

3 第 10 题让编程智能体进入可验证的修复循环

题目要求

- 复制 q09 为 q10, 添加一个失败测试: 当 name 只含空白字符时, main 应以 `SystemExit(2)` 结束。
- 先运行测试确认失败, 再向编程智能体说明目标、约束和测试命令, 要求它修改实现并运行测试。
- 人工检查 diff, 撤销无关修改, 并再次运行测试。
- 在 `ai_log.md` 中用不超过 5 行记录核心提示、智能体改动和人工验证。

操作步骤与命令

```
cp -r q09 q10
mkdir -p q10/tests
vim q10/tests/test_cli.py
cd q10
pytest tests/test_cli.py -v
# 向编程智能体说明目标、约束与测试命令, 由智能体修改 src/greetlab/cli.py
git diff
pytest tests/test_cli.py -v
vim ai_log.md
```

结果验证

- 新增的 `test_blank_name_exits` 初次运行失败 (`DID NOT RAISE SystemExit`);
- 智能体仅修改了 `src/greetlab/cli.py`, 加入 `import sys` 与空白校验, `git diff` 中无无关改动;

- 再次运行 `pytest` 通过，正常姓名输入的行为未受影响；
- `ai_log.md` 用 3 行记录核心提示、智能体改动与人工验证，未超过 5 行。

```
(env) qlao-haiuan@qlao-haiuan-Virtual-Platform:~/system-dev-tools/week3/q10$ python -m pytest tests/ -v
test session starts
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0 -- /home/qlao-haiuan/system-dev-tools/week3/q09/env/bin/python
cachedir: .pytest_cache
rootdir: /home/qlao-haiuan/system-dev-tools/week3/q10
configfile: pyproject.toml
collected 1 item

tests/test_cli.py::test_blank_name_exits FAILED [100%]

===== FAILURES =====
test_blank_name_exits
monkeypatch = <pytest.monkeypatch.MonkeyPatch object at 0x75fddadd5b0>

def test_blank_name_exits(monkeypatch):
    monkeypatch.setattr(sys, "argv", ["sd-greet", "--name", ""])
    with pytest.raises(SystemExit) as exc_info:
        >
E       Failed: DID NOT RAISE SystemExit

tests/test_cli.py:13: Failed
----- Captured stdout call -----
Hello, 
short test summary info
FAILED tests/test_cli.py::test_blank_name_exits - Failed: DID NOT RAISE SystemExit
1 failed in 0.04s
```

图 3: 添加失败测试后初次运行 `pytest` 截图

```
PS D:\Storage\work\SDTreports\system-dev-tools\week3\q10> git diff
diff --git a/week3/q10/src/greetlab/cli.py b/week3/q10/src/greetlab/cli.py
index 99969f9..ea6081f 100644
--- a/week3/q10/src/greetlab/cli.py
+++ b/week3/q10/src/greetlab/cli.py
@@ -1,6 +1,13 @@
 import argparse
+import sys
+
+def main():
+    p = argparse.ArgumentParser()
+    p.add_argument("--name", required=True)
+    a = p.parse_args()
-    print(f"Hello, {a.name}!")
+    if not a.name.strip():
+        sys.exit(2)
+    print(f"Hello, {a.name}!")
\ No newline at end of file
PS D:\Storage\work\SDTreports\system-dev-tools\week3\q10>
```

图 4: 智能体改动后的 `git diff` 截图

```
(env) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week3/q105 python -m pytest tests/ -v
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0 -- /home/qiao-haixuan/system-dev-tools/week3/q09/env/bin/python
cachedir: .pytest_cache
rootdir: /home/qiao-haixuan/system-dev-tools/week3/q10
configfile: pyproject.toml
collected 1 item

tests/test_cli.py::test_blank_name_exits PASSED [100%]

===== 1 passed in 0.01s =====
```

图 5: pytest 通过截图

4 第 11 题把“无法处理”的协作材料改成可执行信息

题目要求

- 在 communication.md 中重写 Issue，包含环境、复现命令、期望结果和实际结果；未知信息写“待确认”。
- 重写提交信息：标题使用祈使语气，正文说明问题和解决方案。
- 重写评审意见：指出具体行为、风险和建议动作，并标注 Blocking、Suggestion 或 Nit。
- 全文控制在 400 字以内，保持专业、简洁。

操作步骤与命令

```
vim communication.md
wc -m communication.md
```

结果验证

- Issue 部分包含环境（操作系统“待确认”、Python 版本“待确认”、包版本 greetlab_25090012035-0.1.0）、复现命令 `sdt-greet --name "`”、期望结果与实际结果；
- 提交信息标题为祈使语气 `fix(cli): reject blank names with non-zero exit code`，正文分别说明问题与解决方案；
- 评审意见指出具体行为（未校验 `-name`）、潜在风险（退出码掩盖调用错误）与建议动作，并标注 **Blocking**；

- 全文 400 字以内，无“尽快修复”“写得不好”等无法执行的信息。

5 第 12 题修复一个可复现的线性回归训练循环

题目要求

- 补全训练循环，正确调用 `zero_grad`、`backward` 和 `step`。
- 训练后调用 `model.eval()`，并在 `torch.no_grad()` 中计算最终损失。
- 打印最终损失、`weight` 和 `bias`；最终损失应小于 0.001。
- 不得直接把 `weight` 和 `bias` 赋值为 3 和 -1。

操作步骤与命令

```
vim train.py
python train.py
```

结果验证

- 训练循环中每个迭代依次调用 `opt.zero_grad()`、`loss.backward()`、`opt.step()`，顺序正确；
- 训练结束后调用 `model.eval()`，并在 `with torch.no_grad():` 中计算最终损失；
- 打印出的最终损失小于 0.001，`weight` 与 `bias` 分别收敛到接近 3 和 -1 的值；
- 代码中未对 `weight` 和 `bias` 直接赋值，两个参数完全由梯度下降更新得到。



```
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week3/q12$ python train.py
Final Loss: 0.000000
Weight: 3.0000, Bias: -1.0000
```

图 6: `train.py` 代码与运行输出截图